Deliverable 4: Stack Justification

Why these tools, and why not C3.ai or Palantir Foundry

Mills-Operation · AI Reliability Copilot for Hot Mill Operations

Stack Justification

What I used, and why

Python with pandas and scikit-learn for the data generator and anomaly detector. I didn't reach for anything heavier because the actual hard part of this assignment isn't the ML, it's proving the detection idea works at all on a realistic failure signature. scikit-learn gave me a real, trained, evaluated model in an afternoon instead of a week of infrastructure, first IsolationForest, later swapped for LocalOutlierFactor after `notebooks/model_comparison.ipynb` showed it winning a direct bake off (see `docs/ai-partnership-log.md` 's addendum). Same reasoning either way: if I'd spent the week wiring up a deep-learning training pipeline instead, I'd have had a fancier stack and a worse answer to "does this actually catch failures early."

Streamlit first, then a proper React and TypeScript frontend with a FastAPI backend. I built the first working UI in Streamlit on purpose: it gets you from a scored dataframe to a working screen in an afternoon, which is exactly what you want when the open question is still "does the detection approach work at all," not "does the UI look production-ready." Once the model was proven out (11/11 detection on held-out data, a real evaluated threshold tradeoff), I rebuilt the interface properly. A shift supervisor's actual tool needs to feel trustworthy, and a script re-running top to bottom on every click isn't the same thing as a real application with an API boundary. The rebuild is a FastAPI backend that serves the scored data and proxies the copilot call, plus a React and TypeScript frontend (Vite, Tailwind, Recharts) for the supervisor-facing surface. That two-stage sequence, prototype fast to validate the idea, then invest in the real interface once it's proven, is a deliberate engineering decision, not scope creep or a wasted first attempt.

Claude (Anthropic API, Haiku 4.5) for the explanation layer. I write prompts for a living in the sense that most of my recent professional work has been RAG and multi-agent systems, so the thing I actually cared about here wasn't "can an LLM write sentences." It was making sure the prompt explicitly constrains the model to the numbers I hand it (no inventing readings, no inventing timeframes) and has a real fallback path when the call fails. That's the same discipline I'd apply to any production LLM integration, just applied to a five-day prototype instead of a quarter-long project.

Playwright and GitHub Actions instead of Tosca, K6, or Azure DevOps. I don't have access to Nucor's actual Azure DevOps org or Tosca licenses, so I used the closest accessible equivalents and built the CI workflow to mirror what a real pipeline would do: generate data, train, evaluate, test, gate on all of it. The shape of the pipeline is the point, not the specific vendor. Swapping this onto actual Azure DevOps would be a config change, not a redesign.

Why not C3.ai or Palantir Foundry

Nucor's real answer here is probably "we already have one of these, use it," and for a *production* version of this tool, I think that's right. But building the prototype itself on C3.ai or Foundry would have been the wrong call, for reasons that are worth being specific about rather than hand-waving.

Platform ceremony versus proving the idea. Both platforms are built around upfront modeling investment before you get to the interesting part: C3.ai's Type and Blueprint system, Foundry's ontology layer. That investment pays off when you're integrating dozens of data sources across an enterprise and need governed, reusable models of "what is a roll stand" shared across teams. For a one-week exercise where the real question is "does this specific anomaly-detection approach work on this specific failure mode," that ceremony would have eaten most of the week before I'd written a single line of detection logic.

C3.ai specifically already sells this. C3 AI Reliability is a purpose-built predictive-maintenance product, this exact problem category, packaged. Building this prototype on top of it would have meant testing *C3.ai's* model and *C3.ai's*

assumptions about failure signatures, not mine. That defeats the point of an assignment that's explicitly scoring my judgment about the problem, not my ability to configure a vendor product. I'd rather show the actual thinking (which signals matter, why the first threshold was wrong, why the rolling-mean baseline was misleading) than hide it behind a platform's built-in model.

No access, and that's fine. I don't have credentials to Nucor's instance of either platform, so this was partly moot, but I'd have made the same call even with access. Prototype fast and cheap to find out if the idea has legs, then re-platform onto the governed enterprise system once it's proven the idea is worth the integration investment. Building on the heavy platform first, before knowing if the underlying approach even works, is optimizing the wrong variable at the wrong time.

Where a production version would actually live. If this proved out, I'd expect the real version to become a registered model inside whichever platform Nucor standardizes on: data ingestion through Foundry's industrial data integration or C3.ai's Type system pulling from the plant historian, the model itself sitting in that platform's governance layer (access control, audit, versioning, retraining triggers), and the supervisor-facing surface embedded in whatever control-room tooling already exists rather than a standalone web app. The five-day version's job was to earn that investment, not skip straight to it.